

Práctica 1

Lenguajes Formales y de Programación · 2S2026

Manual Técnico

Torneo de Sudoku — Numerix Academy

Nombre: Diego Alejandro Barrios Gomez

Carnet: 201900158

Contenido

Contenido	2
Descripción general.....	3
Arquitectura del sistema.....	4
Diagrama de dependencias	4
Responsabilidad de cada archivo.....	4
Clases implementadas (POO)	5
3.1 Clase Tablero	5
3.2 Clase Jugador.....	5
3.3 Clase Intento.....	5
Lógica de validación matricial	7
4.1 Reconstrucción de la matriz	7
4.2 Extracción de las tres unidades.....	7
4.3 Verificación de una unidad	7
4.4 Verificación de las pistas originales	8
4.5 Cálculo del porcentaje de validez	8
4.6 Diagrama de flujo de la calificación.....	9
Lectura y procesamiento de archivos.....	10
5.1 Estrategia.....	10
5.2 Funciones de validación	11
5.3 Validaciones por archivo	11
5.4 Integridad referencial.....	11
5.5 Estructuras de datos utilizadas	11
Cálculo de métricas.....	12
Funciones de formato.....	12
Ordenamiento de datos.....	13
Generación de reportes HTML.....	14
Manejo de errores.....	15
Estructuras de control utilizadas	15
Requerimientos técnicos	15

Descripción general

El sistema es un programa de consola desarrollado en Python 3 que funciona como motor de calificación y análisis de un torneo de Sudoku. Lee tres archivos de texto delimitados por comas (.lfp), reconstruye cada tablero como una matriz de 9×9, valida cada intento contra las reglas del Sudoku, calcula métricas de desempeño y produce reportes analíticos en HTML.

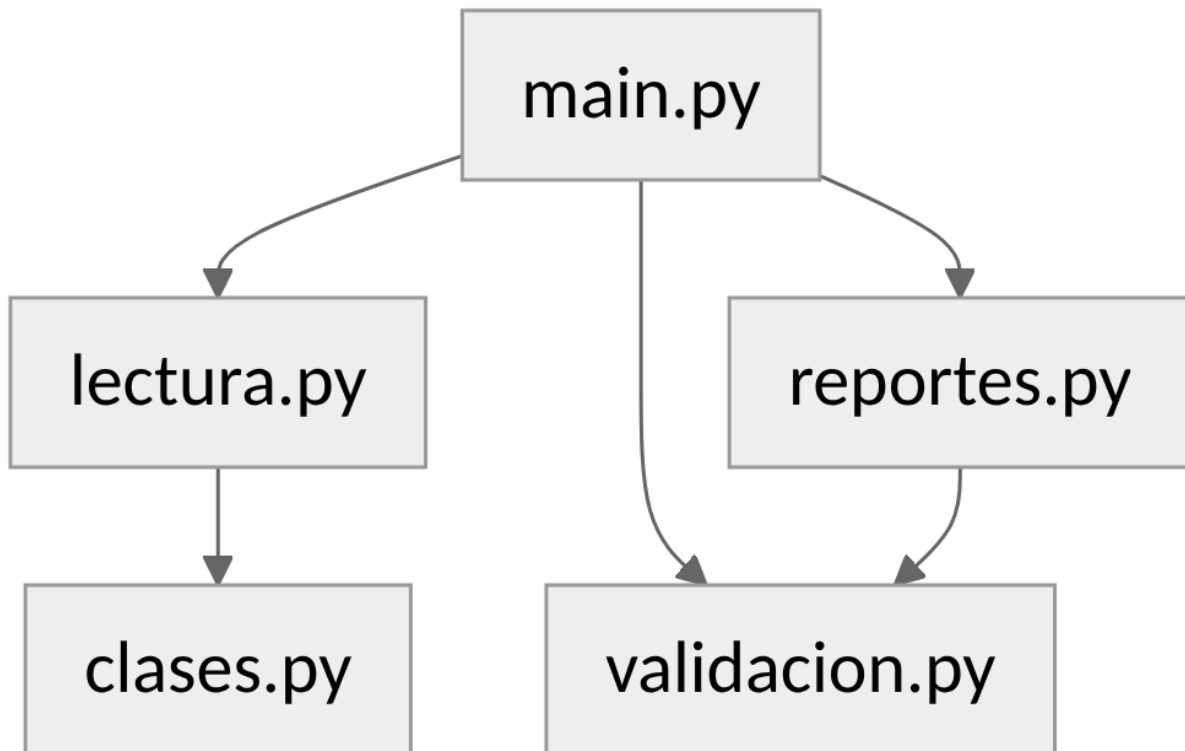
Arquitectura del sistema

El proyecto está organizado en cinco archivos .py ubicados en la carpeta principal, sin subcarpetas de código. Cada archivo agrupa las funciones de una misma responsabilidad.

Practica1/

- |— main.py Menú de consola y control del flujo
- |— clases.py Clases Tablero, Jugador e Intento
- |— lectura.py Lectura de archivos .lfp y validación de formato
- |— validacion.py Validación matricial del Sudoku
- |— reportes.py Cálculo de estadísticas y generación de HTML
- |— entrada/ Archivos .lfp de entrada
- |— reportes/ Salida .html generada

Diagrama de dependencias



Responsabilidad de cada archivo

Archivo	Contiene
clases.py	Los tres moldes de objetos y la función que convierte una cadena de 81 dígitos en matriz 9×9
lectura.py	Apertura de archivos, separación de campos, validación de formato y creación de objetos
validacion.py	Extracción de filas, columnas y cajas; verificación de la regla; calificación de intentos; búsqueda en listas
reportes.py	Promedios, tasas de éxito, ordenamientos y construcción de los archivos HTML
main.py	Menú, estado del programa y llamadas a las funciones anteriores

Clases implementadas (POO)

3.1 Clase Tablero

Representa un tablero publicado en el torneo.

Atributo	Tipo	Descripción
id_sudoku	int	Identificador único
dificultad	str	Facil, Media, Dificil o Experto
cadena	str	Cadena original de 81 dígitos
matriz	list de list	Matriz 9×9 reconstruida

Método	Descripción
__init__(id_sudoku, dificultad, cadena)	Guarda los datos y construye la matriz
contar_pistas()	Recorre las 81 celdas y cuenta las distintas de 0
dibujar()	Devuelve el tablero como texto, con separadores de cajas

3.2 Clase Jugador

Atributo	Tipo	Descripción
carnet	int	Identificador único
nombre, apellido	str	Datos personales
nivel	str	Principiante, Intermedio o Experto

Método	Descripción
obtener_nombre_completo()	Concatena nombre y apellido con un espacio

3.3 Clase Intento

Combina los datos leídos del archivo con los resultados de la calificación, que se llenan después al ejecutar la opción 4.

Atributo de entrada	Descripción
carnet, id_sudoku	Referencias hacia un Jugador y un Tablero
solucion	Cadena de 81 dígitos
matriz	Matriz 9×9 de la propuesta
tiempo_segundos, fecha	Datos del registro

Atributo de resultado	Valor inicial	Descripción
filas_validas	0	Filas correctas (0–9)
columnas_validas	0	Columnas correctas (0–9)
cajas_validas	0	Cajas 3×3 correctas (0–9)
porcentaje_validez	0.0	Porcentaje sobre 27 unidades

pistas_respetadas	True	Si no modificó celdas fijas
celdas_modificadas	[]	Lista de pares [fila, columna]
resuelto_correctamente	False	Veredicto final
observacion	"Sin calificar"	Explicación textual del resultado

Método	Descripción
contar_unidades_validas()	Suma filas, columnas y cajas correctas

Lógica de validación matricial

4.1 Reconstrucción de la matriz

La función `convertir_cadena_a_matriz(cadena)` de `clases.py` transforma los 81 caracteres en una lista de listas. La correspondencia entre la posición lineal y las coordenadas es:

```
posicion = numero_fila * 9 + numero_columna
```

4.2 Extracción de las tres unidades

Una **unidad** es una fila, una columna o una caja de 3×3. Hay 27 en total.

FUNCION `obtener_fila(matriz, numero_fila)`:

```
fila ← lista vacía
PARA numero_columna DESDE 0 HASTA 8 HACER
    fila.agregar( matriz[numero_fila][numero_columna] )
RETORNAR fila
```

FUNCION `obtener_columna(matriz, numero_columna)`:

```
columna ← lista vacía
PARA numero_fila DESDE 0 HASTA 8 HACER
    columna.agregar( matriz[numero_fila][numero_columna] )
RETORNAR columna
```

FUNCION `obtener_caja(matriz, numero_caja)`:

```
fila_inicial ← (numero_caja DIV 3) * 3
columna_inicial ← (numero_caja MOD 3) * 3
caja ← lista vacía
PARA i DESDE fila_inicial HASTA fila_inicial + 2 HACER
    PARA j DESDE columna_inicial HASTA columna_inicial + 2 HACER
        caja.agregar( matriz[i][j] )
RETORNAR caja
```

Las cajas se numeran de 0 a 8, de izquierda a derecha y de arriba hacia abajo:

```
+-----+-----+-----+
| caja0 | caja1 | caja2 |
+-----+-----+-----+
| caja3 | caja4 | caja5 |
+-----+-----+-----+
| caja6 | caja7 | caja8 |
+-----+-----+-----+
```

El cálculo de la esquina superior izquierda usa división entera y residuo. Para la caja 5:

```
fila_inicial = (5 // 3) * 3 = 1 * 3 = 3
columna_inicial = (5 % 3) * 3 = 2 * 3 = 6
```

4.3 Verificación de una unidad

Una unidad es válida si contiene los dígitos del 1 al 9 y ninguno se repite. La implementación cuenta las apariciones de cada dígito:

FUNCION `unidad_es_valida(valores)`:

```
PARA numero_buscado DESDE 1 HASTA 9 HACER
    veces ← 0
```

```

PARA CADA valor EN valores HACER
  SI valor = numero_buscado ENTONCES
    veces ← veces + 1
FIN PARA
SI veces ≠ 1 ENTONCES
  RETORNAR Falso
FIN PARA
RETORNAR Verdadero

```

Esta única revisión cubre tres situaciones distintas:

Situación	Cómo se detecta
Un número repetido	Ese número aparece 2 o más veces
Un número faltante	Ese número aparece 0 veces
Una celda vacía (0)	El 0 desplaza a algún dígito del 1 al 9, que entonces aparece 0 veces

4.4 Verificación de las pistas originales

Se recorren las 81 celdas comparando el tablero publicado contra la propuesta. Si el tablero traía un valor fijo y la propuesta tiene otro, esa celda se registra como modificada.

```

FUNCION buscar_pistas_modificadas(tablero, intento):
  celdas_modificadas ← lista vacía
  PARA numero_fila DESDE 0 HASTA 8 HACER
    PARA numero_columna DESDE 0 HASTA 8 HACER
      valor_original ← tablero.matriz[numero_fila][numero_columna]
      valor_del_jugador ← intento.matriz[numero_fila][numero_columna]
      SI valor_original ≠ 0 ENTONCES
        SI valor_del_jugador ≠ valor_original ENTONCES
          celdas_modificadas.agregar( [numero_fila, numero_columna] )
  RETORNAR celdas_modificadas

```

4.5 Cálculo del porcentaje de validez

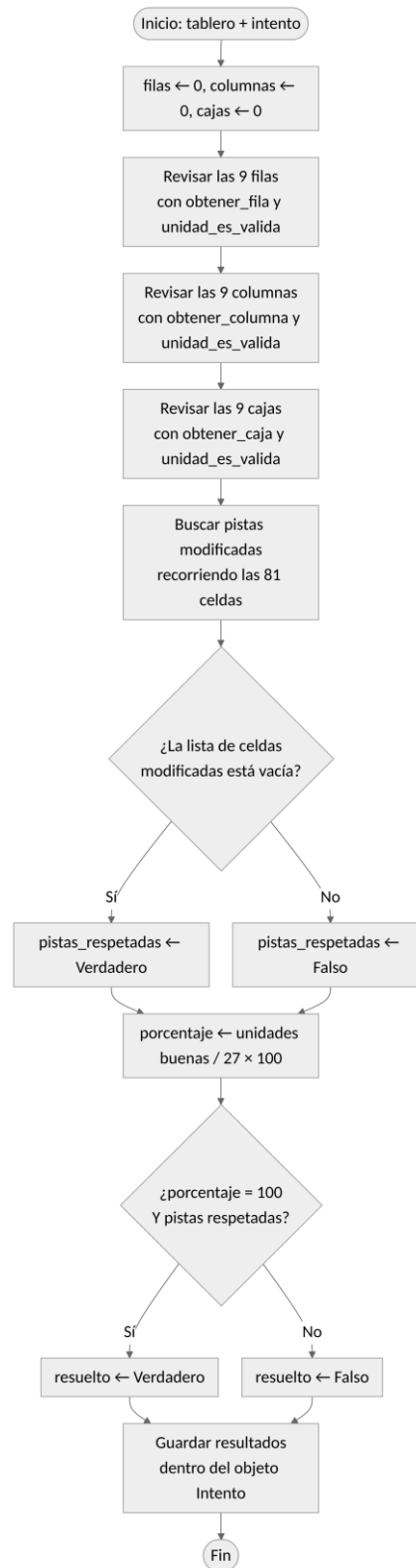
$\text{porcentaje_validez} = (\text{filas válidas} + \text{columnas válidas} + \text{cajas válidas}) / 27 \times 100$

Un intento se marca como **resuelto correctamente** solo si cumple **ambas** condiciones, que son independientes entre sí:

1. `porcentaje_validez == 100`
2. `pistas_respetadas == True`

Esta distinción importa: un jugador puede entregar un tablero perfectamente válido según las reglas del Sudoku pero que no corresponde al que se le asignó, porque modificó las pistas. En ese caso el porcentaje puede ser alto o incluso 100, pero el intento no cuenta como resuelto.

4.6 Diagrama de flujo de la calificación



Lectura y procesamiento de archivos

5.1 Estrategia

Cada función de carga (`cargar_sudokus`, `cargar_jugadores`, `cargar_intentos`) devuelve **dos listas**: los objetos que se pudieron crear y los mensajes de error de las líneas defectuosas.

FUNCION `cargar_X(ruta)`:

`lista_objetos` ← vacía

`lista_errores` ← vacía

`lineas` ← `leer_lineas_del_archivo(ruta)`

 SI `lineas` = Nulo ENTONCES

`lista_errores.agregar("No se encontro el archivo")`

 RETORNAR `lista_objetos`, `lista_errores`

`numero_linea` ← 0

 PARA CADA `linea` EN `lineas` HACER

`numero_linea` ← `numero_linea` + 1

`texto` ← `linea.strip()`

 SI `texto` está vacío ENTONCES continuar

`partes` ← `texto.split(",")`

 SI cantidad de campos incorrecta ENTONCES

`lista_errores.agregar(mensaje); continuar`

 SI algún tipo de dato inválido ENTONCES

`lista_errores.agregar(mensaje); continuar`

 SI dominio cerrado no cumple ENTONCES

`lista_errores.agregar(mensaje); continuar`

 SI identificador duplicado ENTONCES

`lista_errores.agregar(mensaje); continuar`

`objeto` ← crear el objeto correspondiente

`lista_objetos.agregar(objeto)`

 FIN PARA

 RETORNAR `lista_objetos`, `lista_errores`

La instrucción `continue` es la clave ya que al detectar un problema se salta a la siguiente línea en lugar de detener la lectura. Así una línea corrupta no inutiliza todo el archivo.

Los errores se guardan como cadenas de texto, no como excepciones. La razón es que ninguno de estos casos es una falla del programa: son datos de entrada imperfectos, un resultado esperado que la función debe reportar.

5.2 Funciones de validación

Todas están en `lectura.py` y devuelven `True` o `False`:

Función	Verifica
<code>es_numero_entero(texto)</code>	Que el texto no esté vacío y contenga solo dígitos (<code>isdigit()</code>)
<code>dificultad_es_valida(dificultad)</code>	Que sea Fácil, Media, Difícil o Experto
<code>nivel_es_valido(nivel)</code>	Que sea Principiante, Intermedio o Experto
<code>cadena_de_81_digitos(cadena)</code>	Longitud exacta de 81 y que cada carácter sea dígito
<code>fecha_es_valida(fecha)</code>	Formato DD-MM-AAAA, con día 1–31 y mes 1–12
<code>existe_tablero_con_id(lista, id)</code>	Que el identificador no esté repetido
<code>existe_jugador_con_carnet(lista, carnet)</code>	Que el carnet no esté repetido

5.3 Validaciones por archivo

Archivo	Validaciones aplicadas
<code>sudokus.lfp</code>	3 campos; <code>id_sudoku</code> entero; dificultad dentro del dominio; tablero de 81 dígitos; id no duplicado
<code>jugadores.lfp</code>	4 campos; carnet entero; nombre y apellido no vacíos; nivel dentro del dominio; carnet no duplicado
<code>intentos.lfp</code>	5 campos; carnet e <code>id_sudoku</code> enteros; solución de 81 dígitos; <code>tiempo_segundos</code> entero; fecha DD-MM-AAAA

5.4 Integridad referencial

Además del formato, al calificar se verifica que cada intento apunte a un carnet y a un `id_sudoku` que realmente existan. Los que no cumplen se descartan y se listan por separado.

Esta validación es distinta de las anteriores porque **crusa información entre archivos**, y solo puede hacerse cuando las tres listas ya están en memoria. Al leer `intentos.lfp` todavía no se sabe si los otros archivos fueron cargados.

5.5 Estructuras de datos utilizadas

Colección	Estructura	Acceso
Tableros	Lista de objetos Tablero	Función <code>buscar_tablero(lista, id_sudoku)</code>
Jugadores	Lista de objetos Jugador	Función <code>buscar_jugador(lista, carnet)</code>
Intentos	Lista de objetos Intento	Recorrido completo con <code>for</code>
Estadísticas	Listas de diccionarios	Acceso por clave: <code>datos["tasa_exito"]</code>

Las funciones de búsqueda recorren la lista comparando el identificador y devuelven el objeto encontrado, o `None` si no existe.

Cálculo de métricas

Implementado en `reportes.py`.

Métrica	Fórmula	Función
Promedio	suma / cantidad	<code>calcular_promedio(lista)</code>
Tasa de éxito	resueltos / total × 100	<code>calcular_tasa_de_exito(resueltos, total)</code>

Ambas funciones devuelven 0 cuando la colección está vacía, para evitar la división por cero que detendría el programa con un error.

Dificultad real

Tasa de éxito	Dificultad observada
≥ 80%	Facil
≥ 60%	Media
≥ 40%	Dificil
< 40%	Experto

Funciones de formato

Función	Propósito
<code>formato_porcentaje(numero)</code>	Devuelve el número con dos decimales y %. Necesaria porque <code>round(50.0, 2)</code> produce 50.0, con un solo decimal
<code>convertir_a_minutos(segundos)</code>	Convierte segundos a mm:ss usando división entera y residuo
<code>clasificar_dificultad_real(tasa)</code>	Traduce la tasa de éxito a una etiqueta de dificultad observada

Ordenamiento de datos

Los reportes requieren mostrar la información ordenada. Se implementó el **método de la burbuja**, escrito de forma explícita en tres funciones:

Función	Ordena	Criterio
ordenar_por_validez_de_mayor_a_menor(lista)	Diccionarios de jugadores	Validez promedio, descendente
ordenar_intentos_por_tiempo(lista)	Objetos Intento	Tiempo en segundos, ascendente
ordenar_por_tasa_de_menor_a_mayor(lista)	Diccionarios de tableros	Tasa de éxito, ascendente

Funcionamiento del método de la burbuja:

PARA vuelta DESDE 0 HASTA cantidad - 1 HACER

 PARA posicion DESDE 0 HASTA cantidad - 2 - vuelta HACER

 actual $\leftarrow$ lista[posicion]

 siguiente $\leftarrow$ lista[posicion + 1]

 SI están en el orden equivocado ENTONCES

 lista[posicion] $\leftarrow$ siguiente

 lista[posicion + 1] $\leftarrow$ actual

Se compara cada elemento con el que le sigue y se intercambian si están al revés. Después de la primera vuelta, el elemento mayor queda al final; por eso el ciclo interno recorre uno menos en cada vuelta (cantidad - 1 - vuelta).

Las funciones modifican la lista recibida directamente, sin devolver una nueva, porque en Python las listas se pasan por referencia.

Generación de reportes HTML

Un archivo HTML es texto plano. Cada función de reporte construye una cadena larga concatenando etiquetas, y la guarda con extensión .html.

Función	Propósito
armar_documento_html(titulo, subtitulo, contenido)	Envuelve el contenido en la estructura básica de una página HTML
armar_encabezado_tabla(lista_de_titulos)	Construye la primera fila de la tabla con etiquetas <th>
guardar_archivo_html(carpeta, nombre, contenido)	Crea la carpeta si no existe y escribe el archivo en UTF-8

Archivo generado	Contenido
reporte_resumen_sudokus.html	Métricas por tablero
reporte_rendimiento_jugadores.html	Métricas por jugador
reporte_top_tiempos.html	Top 10 mejores tiempos
reporte_sudokus_dificiles.html	Tableros por menor tasa de éxito
reporte_analisis_nivel.html	Desempeño agrupado por nivel

Los reportes usan únicamente etiquetas básicas de HTML, sin hojas de estilo:

Etiqueta	Función
<h1>	Título del reporte
<p>	Párrafos del subtítulo y el pie
<table border='1'>	Tabla con las líneas de la cuadrícula dibujadas por el navegador
<tr>	Una fila de la tabla
<th>	Celda de encabezado; el navegador la muestra en negrita
<td>	Celda de datos
<hr>	Línea horizontal antes del pie de página

El atributo `border='1'` de la etiqueta `<table>` es lo que dibuja la cuadrícula.

Manejo de errores

Situación	Tratamiento	Dónde
Archivo inexistente	<code>os.path.exists()</code> devuelve False; la función retorna None y el mensaje se agrega a la lista de errores	<code>leer_lineas_del_archivo()</code>
Línea con formato inválido	Se agrega un mensaje a <code>lista_errores</code> y se usa <code>continue</code> para saltar a la siguiente	Las tres funciones de carga
Carnet o sudoku inexistente	El intento se descarta al calificar y se reporta aparte	<code>opcion_calificar()</code> en <code>main.py</code>
Archivos no cargados antes de calificar	Se revisa el largo de las listas y se muestra un aviso	<code>opcion_calificar()</code>
Intentos no calificados antes de un reporte	Se revisa el largo de <code>intentos_calificados</code> y se muestra un aviso	<code>opcion_generar_reporte()</code>
Opción de menú no numérica	Se compara contra las opciones válidas con <code>if / elif</code>	<code>main()</code>
División entre cero en promedios	Se verifica el largo de la lista antes de dividir	<code>calcular_promedio()</code>

Estructuras de control utilizadas

Estructura	Ejemplo de uso en el proyecto
<code>while True</code>	Ciclo principal del menú, hasta seleccionar la opción 0
<code>while con condición</code>	Relleno de espacios y ceros para alinear texto
<code>for con range</code>	Recorrido de índices 0 a 8 para filas, columnas y cajas
<code>for sobre listas</code>	Recorrido de intentos, jugadores y líneas del archivo
<code>for anidado</code>	Construcción de la matriz, revisión de pistas, método de la burbuja
<code>if / elif / else</code>	Despacho de opciones del menú y clasificación de dificultad
<code>continue</code>	Saltar líneas vacías o con formato inválido
<code>break</code>	Cortar el Top 10 al llegar al décimo elemento

Requerimientos técnicos

Requerimiento	Versión / detalle
Lenguaje	Python 3.8 o superior
Librerías externas	Ninguna
Módulos estándar usados	<code>os</code>
Codificación de archivos	UTF-8
Estándar de estilo	PEP 8 (verificado con <code>pycodestyle</code> , límite de 100 columnas)
Entorno de prueba	Python 3.12 · Windows 10/11 y Linux